

EdgeMed Secure

Threat Model — STRIDE Analysis

Version 1.0 | June 2026 | Classification: Confidential

Author: James Smith | Phase 0 Deliverable

6	5	24	HIGH
STRIDE Categories	Protected Assets	Identified Threats	Overall Risk Rating

Document Control

Document Title	EdgeMed Secure — Threat Model (STRIDE Analysis)
Version	1.0 — Initial Release
Date	June 2026
Author	James Smith
Classification	Confidential — Not for public distribution
Status	Phase 0 Deliverable — Living Document (update each phase)
Next Review	End of Phase 1 (approx. Month 6)
Framework	STRIDE (Microsoft Threat Modeling Framework)
References	NIST SP 800-213, FDA 2023 Cybersecurity Guidance, IEC 62443

Purpose: This document identifies and analyzes security threats to the EdgeMed Secure healthcare edge virtualization platform using the STRIDE threat modeling framework. It defines protected assets, threat actors, attack vectors, and mitigations. This document is a living artifact and must be updated at the start of each development phase.

1. System Overview

EdgeMed Secure is a lightweight security hypervisor designed to run on constrained medical device hardware. Its core function is to isolate a certified legacy OS (the medical application VM) from a modern security and management layer, without requiring FDA re-certification of the certified software stack. The platform provides VM isolation, inter-VM communication policy enforcement, tamper-evident audit logging,

and TPM-based measured boot attestation.

1.1 Architecture Summary

- Host Layer:** Ubuntu Server 26.04 LTS running KVM/QEMU. Manages VM lifecycle via libvirt. Enforces AppArmor mandatory access control policies on all QEMU processes.
- Medical VM:** Isolated VM containing the certified legacy OS. No outbound network initiation permitted. Receives only defined inbound communications from the management VM via policy engine.
- Management VM:** Modern OS VM running the management API, monitoring agents, and update services. Has limited, policy-defined access to the Medical VM. Can initiate outbound connections for telemetry and updates.
- Audit VM:** Minimal Alpine Linux VM with append-only log storage. Receives syslog from all other VMs over TLS. No outbound network access. SHA-256 chained log entries for tamper detection.
- Policy Engine:** iptables-based inter-VM communication whitelist. All traffic between VM networks is denied by default. Only explicitly defined flows are permitted.
- Attestation Layer:** TPM 2.0 hardware chip. Measures hypervisor and VM boot images into PCR registers. Remote attestation API allows hospital IT to cryptographically verify device integrity.

1.2 Trust Boundaries

The following trust boundaries define where security controls must be enforced:

- TB-1: Between the host hypervisor and each guest VM
- TB-2: Between the Medical VM network and the Management VM network
- TB-3: Between the Management VM and the Audit VM
- TB-4: Between the device and the external hospital network
- TB-5: Between the management API and remote operators
- TB-6: Between the boot process and runtime execution (measured boot boundary)

2. Protected Assets

The following assets must be protected by the platform. Each asset is classified by confidentiality requirement. Integrity and availability requirements apply to all assets.

#	Asset	Description	Confidentiality
1	Certified Medical OS State	The running state and data of the certified legacy medical OS VM. FDA-certified software whose integrity must be guaranteed at all times.	HIGH
2	Patient Data	Any patient-identifiable data processed or stored by the medical application VM. Subject to HIPAA and state privacy regulations.	HIGH

3	Audit Log Integrity	The append-only tamper-evident log of all system events. Critical for regulatory compliance, incident response, and litigation.	HIGH
4	TPM PCR Values / Boot Measurements	Cryptographic measurements of the boot chain stored in TPM PCR registers. Compromise allows undetected tampering with the hypervisor or VM images.	HIGH
5	Management API Credentials	Authentication tokens and keys used to access the management interface. Compromise allows full control of the platform.	HIGH
6	VM Disk Images	The QCOW2 disk images for all VMs stored in /var/lib/libvirt/images/. Tampering can introduce malicious code into any VM.	HIGH
7	Inter-VM Policy Configuration	The iptables rules defining what communication is permitted between VMs. Tampering can eliminate isolation guarantees.	HIGH
8	SSH Private Keys	The ed25519 keys used for administrative SSH access. Compromise allows unauthorized shell access to the host.	MEDIUM
9	GPG Signing Keys	The RSA-4096 keys used for code commit signing. Compromise allows introduction of unsigned or malicious code.	MEDIUM
10	Network Configuration	Virtual network definitions, bridge configurations, and VLAN assignments. Tampering can expose VMs to unintended network segments.	MEDIUM

3. Threat Actors

The following threat actors have been identified as realistic adversaries for a healthcare edge virtualization platform. Threat actors are assessed based on capability, motivation, and likelihood of targeting this specific platform.

Threat Actor	Capability	Motivation	Likelihood
Ransomware Groups	High — automated tooling, known exploits	Financial — encrypt systems, demand payment. Healthcare is #1 target due to urgency of recovery.	HIGH

Nation-State Actors	Very High — zero-days, custom tooling	Intelligence collection, disruption of healthcare infrastructure, supply chain attacks.	MEDIUM
Rogue Vendors	Medium — legitimate access, insider knowledge	A third-party software vendor with a malicious update or agent running in the management VM.	MEDIUM
Insider Threats	Medium — physical and network access	Disgruntled employee or contractor with access to the device or management network.	MEDIUM
Opportunistic Attackers	Low — script kiddies, commodity malware	Automated scanning for known vulnerabilities. No specific targeting of this platform.	HIGH
Medical Device Researchers	High — deep embedded systems knowledge	Security researchers probing for VM escape vulnerabilities or certification bypasses.	LOW

4. STRIDE Threat Analysis

STRIDE is a threat modeling framework developed by Microsoft that categorizes threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Each category is analyzed below with specific threats identified for the EdgeMed Secure platform, risk-rated, and mitigated.

STRIDE — S: Spoofing | Security Property: Authentication

Definition: An attacker impersonates a legitimate user, process, or system component to gain unauthorized access or trust.

ID	Threat Description	Risk	Mitigation
S-01	Attacker spoofs the management API identity to issue commands to the hypervisor host, causing unauthorized VM starts, stops, or configuration changes.	HIGH	JWT token authentication on all API endpoints. HTTPS with certificate pinning. Rate limiting and failed auth logging to audit VM.
S-02	Malicious VM impersonates the audit VM to inject false log entries, covering attacker tracks.	HIGH	Audit VM uses TLS mutual authentication for syslog. SHA-256 log chaining detects any injected entries. Audit VM network is receive-only.
S-03	Attacker performs SSH credential spoofing against the host management interface.	MEDIUM	SSH key-only authentication (ed25519). Password auth disabled. PermitRootLogin disabled. UFW restricts SSH to authorized networks only.
S-04	Supply chain attack — a malicious VM image is substituted for a legitimate one during provisioning.	HIGH	All VM images are SHA-256 hashed at creation. TPM-measured boot detects changes to any image before boot. Hashes stored in immutable audit log.

STRIDE — T: Tampering | Security Property: Integrity

Definition: An attacker modifies data or code without authorization, undermining integrity of the system.

ID	Threat Description	Risk	Mitigation
T-01	Attacker with host OS access modifies the Medical VM disk image to introduce malicious code into the certified OS environment.	CRITICAL	AppArmor profile restricts QEMU process file access to its own disk image only. TPM measured boot detects image modification before next boot. VM image hashes stored in audit log.
T-02	Attacker modifies the iptables inter-VM policy rules to allow unauthorized communication between VM networks.	CRITICAL	Policy configuration is version-controlled in git with signed commits. Any iptables change triggers an audit log entry. Policy engine re-validates rules on each boot.

T-03	Attacker tampers with audit log entries to conceal malicious activity.	HIGH	SHA-256 chain links every log entry to the previous. Any modification breaks the chain. Audit VM has no write-capable network interface. Log storage is append-only at OS level.
T-04	Attacker modifies the hypervisor boot chain (GRUB/initrd) to load a backdoored kernel.	CRITICAL	UEFI Secure Boot with pre-enrolled keys. TPM PCR measurements capture boot chain state. Attestation API alerts if measurements deviate from baseline.
T-05	Attacker modifies the inter-VM policy engine code to insert a policy bypass.	HIGH	All code changes require GPG-signed commits. Policy engine code is hash-verified on startup. Changes trigger audit log entries.

STRIDE — R: Repudiation | Security Property: Non-repudiation

Definition: An attacker performs malicious actions and then denies having done so, exploiting lack of audit trail.

ID	Threat Description	Risk	Mitigation
R-01	Administrator performs unauthorized configuration change and claims it never happened, with no audit trail to refute the denial.	HIGH	All management API calls are logged to the immutable audit VM with timestamp, user identity, source IP, and action taken. Logs cannot be modified or deleted.
R-02	Attacker gains access, performs malicious actions, then clears logs on the compromised VM.	HIGH	Logs are forwarded in real-time to the isolated audit VM over TLS. Local logs on compromised VMs are irrelevant — audit VM holds the authoritative record.
R-03	VM escape goes undetected because no host-level process activity logging exists.	MEDIUM	AppArmor audit mode logs all policy violations. Host-level auditd records all syscalls from QEMU processes. All anomalies forwarded to audit VM.

STRIDE — I: Information Disclosure | Security Property: Confidentiality

Definition: Sensitive information is exposed to unauthorized parties, violating confidentiality.

ID	Threat Description	Risk	Mitigation
I-01	Attacker exploits a VM escape vulnerability to read memory from the Medical VM, exposing patient data.	CRITICAL	KVM memory isolation prevents cross-VM memory access. AppArmor restricts QEMU memory operations. Regular KVM security patch application. QEMU process runs as unprivileged user.

I-02	Network traffic between VMs is intercepted on the virtual bridge, exposing inter-VM communications.	HIGH	All inter-VM communication is encrypted at the application layer (TLS). Virtual bridge traffic is isolated per network. No shared bridge between Medical and Management networks.
I-03	Management API responses leak internal VM configuration, network topology, or credentials.	HIGH	API responses are sanitized — no internal paths, credentials, or topology details returned. API input/output validation enforced. Secrets stored in environment variables, never in API responses.
I-04	Audit logs transmitted over the network expose sensitive operational data to eavesdropping.	MEDIUM	Syslog forwarding to audit VM uses TLS 1.3 encryption. Certificate validation enforced. No plaintext log transmission.
I-05	SSH private keys or GPG keys exposed through accidental commit to version control.	MEDIUM	Git pre-commit hooks scan for private key patterns. Keys stored in ~/.ssh and ~/.gnupg only. .gitignore configured to exclude key files. GitHub secret scanning enabled.

STRIDE — D: Denial of Service | Security Property: Availability

Definition: An attacker disrupts availability of the system or its components, preventing legitimate use.

ID	Threat Description	Risk	Mitigation
D-01	Attacker consumes all available CPU or memory resources from within a compromised VM, starving other VMs including the Medical VM.	CRITICAL	libvirt resource quotas enforce CPU and memory limits per VM. cgroups enforce hard limits. Medical VM gets guaranteed CPU allocation. Resource exhaustion in one VM cannot impact others.
D-02	Ransomware encrypts VM disk images on the host filesystem, rendering all VMs unbootable.	CRITICAL	VM disk images stored with restricted permissions (libvirt-qemu user only). AppArmor prevents unauthorized processes from accessing /var/lib/libvirt/images/. Regular offline snapshots provide recovery point.
D-03	Attacker floods the management API with requests, making it unresponsive to legitimate operators.	MEDIUM	Rate limiting enforced on all API endpoints. Request throttling per source IP. Watchdog service auto-restarts management API if unresponsive. Management plane failure does not affect Medical VM operation.
D-04	libvirtd daemon crash or hang causes loss of VM management capability.	MEDIUM	systemd Restart=always ensures libvirtd auto-restarts. Medical VM continues running independently of libvirtd state. Watchdog monitors daemon health and alerts via audit log.

STRIDE — E: Elevation of Privilege | Security Property: Authorization

Definition: An attacker gains capabilities or access beyond what they are authorized for.

ID	Threat Description	Risk	Mitigation
E-01	Attacker exploits a KVM/QEMU vulnerability to escape from a guest VM and gain host OS access (VM escape).	CRITICAL	QEMU process runs as unprivileged libvirt-qemu user. AppArmor profile strictly limits QEMU capabilities. SELinux/AppArmor sVirt labels isolate each VM's QEMU process. Regular security patching of KVM/QEMU. seccomp filter limits syscalls available to QEMU.
E-02	Compromised Management VM leverages policy engine access to gain control of the Medical VM.	HIGH	Policy engine enforces strict whitelist — Management VM has no shell or administrative access to Medical VM. VM console access requires explicit host-level authentication. libvirt ACLs prevent cross-VM console access.
E-03	Attacker exploits a privilege escalation vulnerability in the Ubuntu host OS to gain root access.	HIGH	Regular OS patching via apt dist-upgrade. Minimal package installation reduces attack surface. sudo access restricted to specific commands. UFW firewall limits network exposure of host services.
E-04	Unprivileged user on the host escalates to libvirt-qemu user to access VM disk images.	HIGH	/var/lib/libvirt/images/ permissions set to 711. Files owned by libvirt-qemu:libvirt. AppArmor prevents unauthorized access. Host user accounts follow least-privilege principle.
E-05	Attacker uses a malicious QEMU device emulation exploit to gain ring 0 execution on the host.	CRITICAL	Only necessary QEMU devices are emulated per VM. Unnecessary device emulation disabled in VM XML configuration. QEMU compiled with hardening flags. seccomp sandbox enabled for QEMU processes.

5. Risk Summary Matrix

The following table summarizes all identified threats by risk level. Critical and High risks must be mitigated before the platform handles real patient data.

Risk Level	Count	Threat IDs	Summary
CRITICAL	6	T-01, T-02, T-04, I-01, D-01, D-02, E-01, E-05	VM image tampering, policy bypass, boot chain compromise, VM escape, patient data exposure, ransomware, resource exhaustion
HIGH	13	S-01,S-02,S-04,T-03,T-05,R-01,R-02, I-02,I-03,D-01,E-02,E-03,E-04	API spoofing, audit log tampering, information disclosure, privilege escalation, unauthorized VM access
MEDIUM	5	S-03,R-03,I-04,I-05,D-03,D-04	SSH spoofing, missing host audit, encrypted log interception, key exposure, API DoS, daemon failure
LOW	0	–	No low-risk threats identified at this stage

6. Mitigation Roadmap

Security controls are implemented across development phases. Controls marked 'Implemented' are active in the current Phase 0/1 baseline. Controls marked Phase 2 or Phase 3 are planned mitigations on the development roadmap.

Phase	Control	Status	Implementation
Phase 0	SSH key-only authentication	Implemented	ed25519 keys, password auth disabled, root login disabled
Phase 0	UFW host firewall	Implemented	Deny by default, port 22/tcp only
Phase 0	GPG signed commits	Implemented	RSA-4096, Verified badge on GitHub
Phase 0	UEFI Secure Boot	Implemented	OVMF with pre-enrolled keys on VM 102
Phase 0	TPM 2.0 hardware present	Implemented	TPM v2.0 provisioned on dev VM
Phase 1	VM network isolation	Phase 1	mednet/mgmtnet separate networks, iptables deny-by-default
Phase 1	libvirt resource quotas	Phase 1	CPU/memory limits per VM via libvirt XML
Phase 1	VM snapshot recovery baseline	Phase 1	virsh snapshot-create-as for each VM

Phase 2	AppArmor QEMU profiles	Phase 2	Custom AppArmor profile restricting each VM's QEMU process
Phase 2	Immutable audit VM	Phase 2	Alpine audit VM, TLS syslog, SHA-256 chained logs
Phase 2	TPM measured boot implementation	Phase 2	tpm2-tools PCR measurements, attestation script
Phase 2	Inter-VM policy engine	Phase 2	JSON policy schema, auto-generated iptables rules
Phase 2	seccomp filter for QEMU	Phase 2	Syscall whitelist for QEMU processes
Phase 3	Management API with JWT auth	Phase 3	FastAPI, HTTPS, rate limiting, audit logging
Phase 3	Remote attestation endpoint	Phase 3	API returns cryptographically verifiable PCR state
Phase 3	Offline-first state sync	Phase 3	SQLite queue, persisted, applies on reconnect
Ongoing	Regular OS and KVM/QEMU patching	Ongoing	apt dist-upgrade on schedule, CVE monitoring
Ongoing	Threat model review each phase	Ongoing	Update this document at start of Phase 2, 3

7. Assumptions and Exclusions

7.1 Assumptions

- The Proxmox host and underlying AMD Threadripper hardware are physically secure in the development environment.
- The KVM hypervisor and QEMU are assumed to be free of currently-known, unpatched vulnerabilities when kept up to date.
- The TPM 2.0 hardware module is assumed to be a trusted root — hardware-level TPM attacks are out of scope for this phase.
- The development network (192.168.1.x) is a trusted home lab network. Production deployments will require additional network-level controls.
- The Ubuntu 26.04 LTS base OS is a trusted starting point when installed from official ISO and kept patched.
- libvirt and its dependencies are assumed trustworthy when sourced from official Ubuntu repositories.

7.2 Out of Scope (This Version)

- Physical hardware attacks against the TPM chip or memory buses.
- Side-channel attacks (Spectre, Meltdown variants) — acknowledged risk, mitigated by CPU microcode updates.
- Attacks against the Proxmox management interface (separate system, not part of EdgeMed Secure).
- Social engineering attacks against the developer or system administrators.
- Attacks against the GitHub repository or CI/CD pipeline (future concern when pipeline exists).
- Formal cryptographic analysis of the SHA-256 log chaining scheme.

8. Regulatory Alignment

The following table maps key mitigations to relevant regulatory requirements. This mapping supports future FDA 510(k) submissions and IEC 62443 certification.

Mitigation	NIST SP 800-213	FDA 2023 Guidance	IEC 62443	HIPAA
VM Isolation	IR-4, SC-39	Section 3.1	SR 3.1	164.312(a)
Measured Boot / Attestation	SI-7, SA-10	Section 5.2	SR 3.3	164.312(c)
Immutable Audit Logging	AU-9, AU-10	Section 7.1	SR 2.8	164.312(b)
Inter-VM Policy Engine	AC-4, SC-7	Section 3.2	SR 3.1	164.312(a)
Encrypted Communications	SC-8, SC-28	Section 4.1	SR 4.1	164.312(e)
TPM-based Integrity	SI-7, IA-3	Section 5.1	SR 3.3	164.312(c)

9. Next Steps & Review Schedule

Immediate (Phase 1): Implement VM network isolation — create mednet and mgmtnet, configure iptables deny-by-default, verify VMs cannot communicate.

Immediate (Phase 1): Configure libvirt resource quotas on all VMs to prevent resource exhaustion attacks.

Phase 2 Start: Implement AppArmor QEMU profiles for each VM. This is the single highest-impact security control.

Phase 2 Start: Deploy immutable audit VM with TLS syslog and SHA-256 log chaining.

Phase 2 Start: Implement TPM measured boot with tpm2-tools. Establish PCR baseline.

Phase 2 Start: Build and test the inter-VM policy engine with JSON schema and auto-generated iptables rules.

Phase 2 Complete: Conduct a simulated attack scenario: compromise one VM, verify containment, verify audit trail.

End of Phase 2: Update this threat model document with new threats identified during implementation.

Phase 3 Start: Implement management API with full authentication, rate limiting, and audit integration.

Ongoing: Monitor NVD (nvd.nist.gov) for new CVEs against KVM, QEMU, libvirt, and Ubuntu kernel.

Document Review Schedule: This threat model must be reviewed and updated at the start of Phase 2 (Month 6), Phase 3 (Month 12), and before any OEM partner engagement or public release. New threat vectors discovered during implementation must be added immediately.